U.T. 9.- ENTRADA/SALIDA EN JAVA

Sesión 1. Introducción.

La comunicación de aplicaciones entre sí, o con el exterior (usuario, periféricos, etc.) se denomina entrada/salida (E/S), o Input/Ouput (I/O) en inglés.

Un programa sirve de poco si no puede obtener datos del exterior ni comunicar sus resultados. Hasta ahora hemos usado como entrada y salida la escritura en consola (pantalla de texto) y la lectura desde teclado. En este tema veremos otras posibilidades.

Nuestros programas se ejecutan cargándose en memoria RAM y ahí manejan todos sus datos (variables, atributos …) como espacio de trabajo. La memoria RAM es volátil, pierde sus datos al apagarse el ordenador. Además, al terminar el programa, la memoria RAM que estaba ocupando se libera, se pierden los datos ahí almacenados pues se reasigna a otros programas.

Necesitamos pues, un espacio de almacenamiento no volátil donde poder dejar datos que recuperar en la siguiente ejecución de un programa. Java nos permitirá, a través del sistema operativo, acceder a los sistemas de archivos de los dispositivos de almacenamiento disponibles.

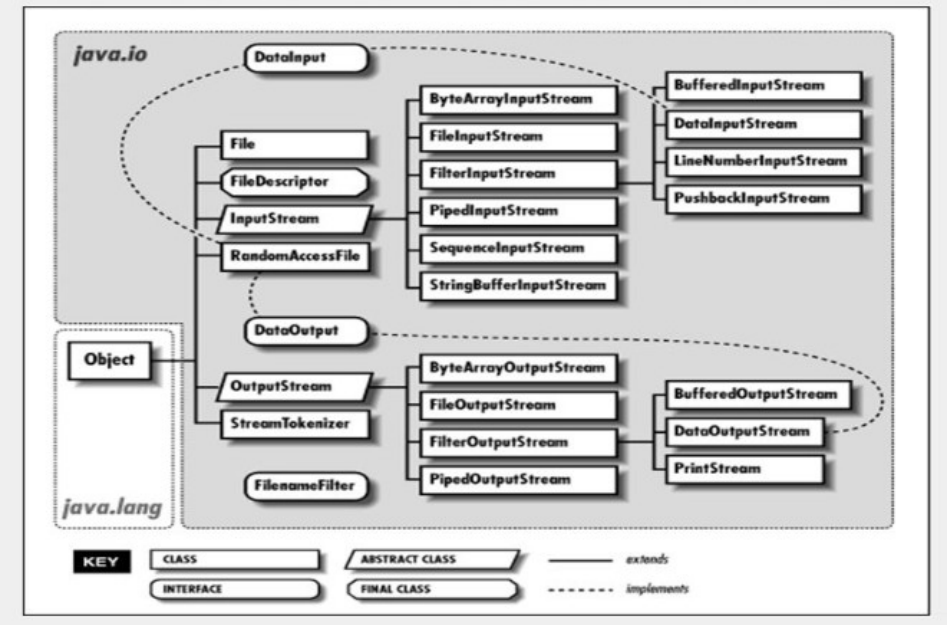
1. Paquetes java.io y java.nio

En Java, las clases que gestionan este aspecto de las aplicaciones están en dos paquetes: **java.io** y **java.nio**. Este curso trabajaremos solo con java.io.

Las clases de estos paquetes son muchas y están relacionadas entre sí mediante los mecanismos de herencia y polimorfismo. Podemos saber de forma sencilla cómo se comporta una clase analizando las palabras que componen su nombre:

- clases Reader: orientadas a la lectura de caracteres (texto)
- clases Writer: orientadas a la escritura de texto
- clases InputStream: orientadas a la lectura de flujo de datos binarios
- clases OutputStream: orientadas a la escritura de datos binarios
- clases Printer: formatean el resultado (métodos printf, println)

Ej: BufferedReader, ObjectInputStream, DataOutputStream...



Algunas clases complementarias a la E/S están en otro paquetes, como la clase **Scanner** pertenece a **java.util**

2. La clase File

Java usará espacios de almacenamiento no volátil a través del sistema operativo, lo hará a través de sistemas de archivos asociados a unidades de disco, SSD u otros. Los sistemas de archivos se acceden mediante rutas que forman árboles de carpetas y archivos con permisos y atributos. Accedemos a estructuras fuera de la máquina virtual de Java, que pueden estar siendo usadas por otros procesos y, por lo tanto, podrá haber situaciones excepcionales que no hayamos contemplado ni podamos anticipar (exceptions)

La clase **File** permite **gestionar NOMBRES y RUTAS** de streams de IO, pero no permite acceder a su contenido. Es decir, un objeto File contiene una **referencia** a un nombre de archivo o a un directorio PERO solo referencia el nombre, no el contenido del archivo.

Además, todas las clases que llevan File en el nombre son descendientes de File y por tanto permite manejar nombres y rutas de forma más especializada: FileOutputStream, FileInputStream, FileReader, …

Las clases que no llevan en su nombre la palabra File (BufferedReader, InputStreamReader, etc) necesitarán “contener” alguna clase File.

Sesión 2.

Podemos crear objetos File con ruta. Si no especificamos ninguna ruta, se crearán **en la carpeta de trabajo (working directory) la raíz del proyecto en el caso de Eclipse o la carpeta desde donde se lance en el caso de la línea de comandos.** Si especificamos alguna ruta, se crearán **siempre que la ruta exista y tengamos permisos de escritura**. Podemos obtener la ruta por defecto (working directory) con System.getProperty("user.dir")

3. Reader y Writer: flujos orientados a carácter (texto)

Reader y Writer son clases abstractas del paquete java.io. Esto significa que podemos instanciar clases que hereden de ellas, pero no podemos instanciarlas a ellas (no se puede tener un objeto tipo Reader).

Las clases tipo Reader tienen un método fundamental **read()** que se comporta de la siguiente forma:

- lee un char, y lo devuelve en forma de int
- cuando ya no hay más por leer, devuelve **-1 (EoF)**

La clase **FileReader** permite leer char a char el contenido de un objeto File. También permite leer de una tacada muchos caracteres pero almacenándolo en un array que tenemos que crear nosotros a modo de buffer, lo cual lo hace más incómodo que son otras clases que vamos a ver a continuación. Además, no separa las líneas por sus retornos de carro, lee bytes sin tener en cuenta la separación de líneas.

Sesión 3.

Ejercicio: Escribir un programa que lea un archivo de texto hola.txt carácter a carácter y sustituya las secuencias \n (LF) por un salto de línea de Windows (\r\n == CR LF).

Conclusión: leer char a char es complejo y da problemas al programador.

Para leer línea a línea, en Java disponemos de la clase **BufferedReader**, que hereda de FileReader y además incorpora métodos útiles, principalmente el método readLine(). Es una de las clases más utilizadas para lectura de texto.

El método readLine() de BufferedReader devuelve una línea como objeto String, y **devuelve null** cuando encuentra EoF.

Para escribir tenemos diferentes clases Writer, entre otras FileWriter o PrintWriter. En este caso, FileWriter sí se usa aunque es simple.

Para escribir en un archivo o flujo en general se usa el método write, que admite no solo chars sino Strings, arrays de bytes…

Al escribir hay que tener en cuenta que, por eficiencia, puede que la clase escritora no escriba inmediatamente y espere a acumular información suficiente para decidirse a escribir.

Para obligar a una clase writer a escribir lo que tiene acumulado, existe el método flush(). El método close() hace flush implícito.

La clase FileWriter creará el archivo de salida si no existe.

Las clases Writer tienen el método write para escribir datos en disco. Por defecto al abrir un archivo para escritura se truncan sus contenidos (se reescribirá totalmente el archivo, empezando de cero) pero también existe la posibilidad de anexar (o añadir == append) los nuevos datos al final del archivo respetando los datos ya existente.

Pare ello el constructor de FileWriter admite como segundo parámetro booleano un True para indicar este comportamiento (append) al igual que muchas otras clases del paquete java.io también (las de escritura en disco)

Sesión 4.

4. Gestión de Strings y flujos de texto

Al utilizar la clase BufferedReader vamos leyendo Strings que contienen una línea de texto completa. En estos casos, es frecuente que necesitemos dividir ese String en “trozos” para extraer palabras o datos que interese procesar separadamente. Todas las opciones están basadas en el uso de expresiones regulares.

Una primera opción es utilizar el método split:

- **toma como argumento** un String que contiene los **caracteres separadores**
- **devuelve un array de String** con los diferentes segmentos separados; cada String es un dato.

Ejemplo:
Escribir un programa que lea datos de personas de “alumnos.csv”, separados por “,”, e instancie un objeto Clase. Deberá insertar cada alumno en la clase con su nota. Finalmente, mostrará el contenido de la Clase.

Ejercicios:
Modificar el ejemplo anterior para que:
1. el listado se escriba en un archivo listado.txt
2. los datos se escriban como tabla en formato html, en un archivo listado.htm
3. los datos de los alumnos suspensos aparezcan en color rojo en la tabla

4. escriba un script alumnos.sql que permita insertar los alumnos en una base de datos con una tabla alumnos.

5. Patrones de caracteres: clases Pattern y Matcher

Una **expresión regular (regex, o regular expression)** es una secuencia de caracteres que simboliza un conjunto de cadenas que cumplen determinadas reglas.

En una expresión regular podemos tener los siguientes símbolos:

- caracteres de texto: a, A, 2, ¡, @,…
- corchetes: permiten definir conjuntos de caracteres [abc] de los cuales se elige uno cualquiera
- paréntesis: permiten agrupar elementos de la expresión regular
- caracteres comodín: “.” significa un carácter cualquiera
- ^ dentro de un rango significa el conjunto complementario de otro (not): [^a-z] es cualquier carácter excepto a-z

Además, un carácter o un conjunto puede llevar una cardinalidad asociada:

- ? : ocurrencia de 0 a 1 (opcionalidad)
- * : ocurrencia de 0 a n
- + : ocurrencia de 1 a n
- {n, m} ocurrencia entre m y n veces
- {n} ocurrencia exacta de n veces
- {n,} ocurrencia n o más veces

Finalmente, hay conjuntos especiales que tienen una notación específica por su frecuencia de uso:

- [a-z]: letras minúsculas
- [A-Z]
- [0-9]
- **\d**: igual que [0-9]
- **\D:** cualquier carácter excepto [0-9]
- **\s**: cualquier separador (espacio en blanco, tabulador, salto de línea…)
- **\S**: cualquier carácter que no sea un separador
- **\w** es un sinónimo de [a-zA-Z0-9] y **\W** del conjunto complementario.

En realidad las barras inversas (\) tendremos que ponerlas por duplicado (ej: \\d) pues dentro de un String la barra inversa es el carácter de escape. Además, para utilizar los caracteres especiales (., ?, ...) no como especiales sino como caracteres “habituales” es necesario “escaparlos” poniendo el carácter “\” delante. Por tanto, “\” también es un carácter especial. Como además en Java \ se utiliza para escapar caracteres especiales en los String, encontraremos que para, por ejemplo, buscar un punto tal cual, nos tocará hacer un doble escape: (“\\\\.”) → se corresponde con la expresión regular \. que busca un punto en vez de interpretar el punto como indicador especial de cualquier carácter.

Ejemplos:

1. Palabras que comiencen por a
a.*

2. Palabras de 5 letras que comiencen por es
es.{3}
es[a-zA-Z]{3}

Para trabajar en Java con expresiones regulares tenemos las clases **Pattern y Matcher**, que pertenecen al **paquete java.util.regex**

Además, la **clase String** tiene un **método matches** que recibe un String regex y decide si el *String* cumple (concuerda, casa, se ajusta) o no la expresión. Y la clase *Pattern* un método estático muy parecido: *Pattern.matches(regexp,texto)*

Ejemplo:

```
String patrónFecha = "[0-9]{2}/[0-9]{2}/[0-9]{4}";
String fecha = entrada.nextLine();
if (fecha.matches(patrónFecha))
        System.out.println("Fecha correcta");
```

// equivalente: `if (Pattern.matches(patrónFecha, fecha))`

nota: observa que las / se buscan tal cual, no son caracteres especiales.

Ejercicios: bien sea con el método de String texto.matches(regexp) o con Pattern.matches(regexp, texto) haz un programa que pida al usuario un texto y compruebe:
1.- Si es un número de teléfono válido.
2.- Si es una matrícula de vehículo correcta.

Sesión 6.

Para **filtrar el contenido de archivos de texto**, vamos a utilizar las clases **Pattern y Matcher**:

- la clase Pattern tiene un método estático para compilar expresiones regulares

Pattern.compile(String regex)

- como el pattern puede ser bastante complejo, usaremos paréntesis () para marcar grupos o segmentos dentro del pattern

- desde un patrón compilado podemos obtener un objeto Matcher

Pattern p= Pattern.compile("(<alumno id= \')(\w*)(\'>)");
Matcher m= p.matcher(String s);

a partir de aquí, podremos escanear el String con los diferentes métodos de la clase Matcher:

- boolean matches(): me dice si ha encontrado el patrón
- String group(int n): me devuelve la subcadena correspondiente al grupo teniendo en cuenta que el primer grupo es el 1 (el 0 es toda la cadena concordada por el patrón)

Ejercicios.
Tenemos un archivo alumnos.xml.
1. Escribir por pantalla un listado con los dnis y nombres de los alumnos.

Sesión 6.

2. Escribir un programa que lea alumnos.xml y por cada alumno que encuentre genere la correspondiente línea de inserción SQL en el archivo alumnos.sql. Tener en cuenta que el elemento <nivel> es opcional en el xml, y que si no aparece tiene un valor por defecto FPB.

Sesión 7.

6.- FLUJOS ORIENTADOS A BYTE

Las clases **InputStream** y **OutputStream** son abstractas y son las raíces de la jerarquía de java.io. Esto significa que todos las clases que trabajan con flujos de datos son descendientes de ellas.

La clase InputStream **no garantiza cuántos bytes se van a leer**, por tanto trabaja con los siguientes métodos:

- int available(): devuelve el número de bytes que hay en el stream o flujo **pendientes de ser leídos**. En un archivo, sería el número de bytes que aún no se han procesado.
- int read(): lee 1 byte y devuelve su codificación a char; devuelve -1 si no hay más datos
- int read(byte[] b): intenta leer tantos bytes como b.length; **devuelve** un int con los **bytes que se han leído realmente**.
- int read(byte[] b, int offset, int len): intenta llenar el buffer b, pero solo a partir de la posición offset y hasta len.

La clase OutputStream tiene métodos similares:

- void write(int b): escribe un byte
- void write(byte[] b): escribe un buffer de bytes
- void write(byte[] b, int offset, int len): escribe todos los bytes de b, desde offset hasta offset+len-1.
- flush(): hay que tener en cuenta que las operaciones de escritura son asíncronas y por tanto cuando hacemos write no podemos garantizar que la información se hay escrito realmente. Es necesario usar flush para obligar a sincronizar la escritura (“commit”).

Siempre que se genera un flujo de salida sobre una archivo que existe, se sobreescribe y se pierden todos los datos que hubiera. Si no queremos que esto ocurra, se puede abrir un flujo con el parámetro append, que concatena los datos manteniendo lo que había.

Ej:
FileOutputStream fos=
new FileOutputStream(String ruta, boolean append);
FileOutputStream fos=
new FileOutputStream("fotos/img.jpg", true);

Ejemplo: copiar un archivo binario teniendo su ruta de acceso

Para **copiar un archivo binario**, se intentan leer los datos rellenando un buffer de una longitud determinada. Sin embargo, el método read no garantiza que el búffer se llene. Por tanto, tendremos en cuenta cuántos bytes se han leído realmente y escribiremos ese mismo número de bytes.

Habitualmente podemos querer guardar conjuntos de datos de diferentes tipos. En estos casos hay dos enfoques: escribir todos los tipos como texto y al recuperarlos, parsear cada tipo (Float.parseFloat(String s), etc.). La otra posibilidad es guardar cada tipo de dato con su tipo, y para esto existen las clases **DataInputStream** y **DataOutputStream**.

Ejemplo: pedir por teclado un dni y una nota en formato x.y hasta que el dni sea 0 o la nota no sea válida (<0 o >10), e ir guardando los datos en un archivo notas.dat

Ejercicio: leer el archivo alumnos.csv y pasar los datos a alumnos.dat, cada uno con su tipo correspondiente. Después, hacer un programa que lea el contenido de alumnos.dat y lo muestre por pantalla.

7.- Flujos de datos y serialización

Apartado “Serialización de objetos” en los apuntes de Fernando Berzal.

8.- Acceso secuencial vs. acceso aleatorio.

Apartado “Ficheros de acceso aleatorio” en los apuntes de Fernando Berzal.

¿API de Logging para otro año?

https://hashblogeando.wordpress.com/2015/08/22/logs-en-java-con-java-util-logging/
https://medium.com/el-acordeon-del-programador/logs-en-java-con-java-util-logging-d344ae2ba7bc
https://docs.oracle.com/javase/10/core/java-logging-overview.htm#JSCOR-GUID-B83B652C-17EA-48D9-93D2-563AE1FF8EDA

Anexo 1. Codificaciones.

Los archivos de texto plano pueden venir con codificaciones distintas. Por un lado tenemos la diferencia en los saltos de línea entre sistemas Windows y derivados de Unix (Android, Linux, MacOs, etc.) En los primeros se utilizan los caracteres CR y LF (10 y 13 en decimal) mientras que en los segundos solo el LF (CR = Carriage Return, LF = Line Feed)

Luego está la codificación en sí de los caracteres. Hoy en día todos los sistemas operativos modernos usan alguna codificación Unicode, siendo UTF-8 la más extendida (Windows, Linux, etc.) pero todavía hay entornos donde se usan variantes del ASCII extendido como ANSI, u otras. De hecho Eclipse en Windows usa por defecto ISO8859-1 (también llamada ISO-LATIN1 para los idiomas de europa occidental) Los códigos de los caracteres pertenecientes al alfabeto americano (ASCII original de 7 bits) son iguales en todas estas, pero para caracteres como letras con tilde, eñe, etc. cambian.

En Java, podemos especificar en las clases de E/S de texto la codificación de los mismos. Si no la especificamos toma por defecto la del sistema o la que hayamos definido en el proyecto (Editar → Propiedades → Resource → Text file encoding) y se refleja en lo que nos dice el método getEncoding() que heredan todos los InputStreamReader y OutputStreamWriter.

En las versiones modernas de Java muchas de las clases que hemos visto (FileReader, FileWriter, etc.) permiten dar la codificación como argumento del constructor. Ejemplo:

```
FileReader fr = new FileReader("entrada.txt", StandardCharsets.ISO_8859_1)
PrintWriter pr = new PrintWriter("salida.txt", StandardCharsets.UTF_8);
```

En versiones muy antiguas lo haríamos construyendo nuestros flujos a partir de InputStreamReader que sí lo permiten. Por ejemplo:

```
BufferedReader br = new BufferedReader(new InputStreamReader(new FileInputStream("pepe.txt"), StandardCharsets.UTF_8));
```

Aunque algunas clases permitan especificar la codificación con un String (ej: “UTF_8”) es más seguro usar las constantes que nos provee el paquete java.nio (Eclipse os sugerirá la importación) y que se han usado más arriba (StandardCharsets.*)

Cuando creáis un archivo de texto plano, la mayoría de los editores (bloc de notas, Notepad++, editor de texto/plano en Linux …) os permiten en “Guardar como” especificar la codificación.

Es un buen **ejercicio** ver los problemas que aparecen cuando leemos o escribimos ficheros con codificación distinta a la esperada. Otro buen ejercicio es hacer un mini-programa que lea un archivo con una codificación y guarde los contenidos en otro archivo con otra codificación.

Para autodetectar la codificación hay que hacer uso de liberías externas: https://stackoverflow.com/questions/499010/java-how-to-determine-the-correct-charset-encoding-of-a-stream
o no:
http://www.java2s.com/Code/Java/I18N/Howtoautodetectafileencoding.htm

Otro tema es el uso de caracteres especiales del alfabeto local en expresiones regulares. La expresión regular "[A-Z]" no incluye letras acentuadas ni eñes. He visto la solución en este artículo, os dejo un ejemplo, aún así hay que usar el atajo \w en vez de [A-Z] sabiendo que \w incluye números:

```
String patrón = "\\w+";
String texto = "áéíóúÁÉÍÓÚÑñ";
//if (texto.matches(patrón)) { // no va
//if (Pattern.matches(patrón,texto)) { // no va
Pattern p =
Pattern.compile(patrón,Pattern.UNICODE_CHARACTER_CLASS);
Matcher m = p.matcher(texto);
if (m.matches()) {
     System.out.println("CONCUERDA");
}
else {
     System.out.println("NO CONCUERDA");
}
```

Leyendo un poco más (este artículo) veo que hay una solución más fácil usando categorías de Unicode y que además, nos vale para buscar solo letras (sin incluir números o guión bajo como con \w) , ahí va:

```
String patrón = "\\p{L}+";
String texto = "áéíóúÁÉÍÓÚÑñ";
if (texto.matches(patrón)) {
...
```

Para aprender más:
https://docs.oracle.com/javase/tutorial/i18n/index.html